

COMP 1111 3. Midterm

Olcaý Taner YILDIZ, Emine EKİN, Ayşegül TÜYSÜZ ERMAN

I. QUESTION (16 POINTS)

Trace the execution of the call `mystery(4)` for the following recursive method by drawing method calls and return values diagram (tracing tree). What does this method do?

```
public static int mystery(int n) {
    if (n == 0)
        return 0;
    else
        return n * n + mystery(n-1);
}
```

II. QUESTION (18 POINTS)

Given an array of integers that are greater than zero as a parameter, write a function called **splitIndex** that checks if it is possible to split this array in two (without changing the order of the elements), such that the sum of the two resulting arrays is the same. The method returns the splitting index (starting index of the second array) if it is possible to split the array; -1 otherwise.

```
public static int splitIndex(int[] A)

A = {5, 2, 3}
Returns 1.
A = {2, 3, 2, 1, 1, 1, 2, 1, 1}
Returns 3.
A = {1, 1, 3}
Returns -1.
A = {1, 1, 3, 1}
Returns -1.
```

III. QUESTION (16 POINTS)

Let say you have the functions **factorial** and **power**

```
public static int factorial(int N)
public static int power(int x, int y)
```

which calculate $N!$ and x^y respectively. Write a function **mysterySum** which takes input an integer k and will return an array containing the following sum for the first k numbers. You must use the factorial and power functions given above. **DO NOT** write the functions **factorial** and **power**.

$$\frac{1!}{2^1} + \frac{2!}{2^2} + \dots + \frac{N!}{2^N}$$

```
public static double[] mysterySum(int k)

k = 1
Returns {0.5}
k = 3
Returns {0.5, 1, 1.75}
k = 5
Returns {0.5, 1, 1.75, 3.25, 7}
```

IV. QUESTION (16 POINTS)

Write a method **prefixSum** to calculate the prefix sum of an integer array passed as parameter and returns the prefix sum in an array. The prefix sum of an array is calculated as following: A is the input array; P is the prefix summed array

```
P[0] = A[0];
P[i] = P[i-1] + A[i]

public static int[] prefixSum(int[] A)

A = {22, 2, 9, 23, 40}
Returns {22, 24, 33, 56, 96}
A = {2, 3, 2, 1, 1, 1, 2, 1, 1}
Returns {2, 5, 7, 8, 9, 10, 12, 13, 14}
```

V. QUESTION (18 POINTS)

Write a Java method, **maxCount**, that takes an integer array, and counts the occurrences of each member of the array. **maxCount** should return the count of the most frequent member. You can assume the elements of the array are non-negative and less than 100.

```
public static int maxCount(int[] A)

A = {6, 2, 4, 5, 6, 2, 3, 6, 8, 4, 6, 8, 6}
Returns 5
A = {2, 3, 2, 1, 1, 1, 2, 1}
Returns 4.
```

VI. QUESTION (16 POINTS)

Let say you have the function **primes**

```
public static int[] primes(int N)
```

which returns all primes $\leq N$. Write a function **printCo-primes** which takes two integers as input M, N and will print the co-prime factors of those two integers. The co-prime factors of two integers are the prime numbers who divide both integers. You must use the primes function given above. **DO NOT** write the function **primes**.

```
public static void printCoprimes(int M, int N)

M = 120, N = 72
2
3
M = 11, N = 121
11
M = 2310, N = 30
2
3
5
```